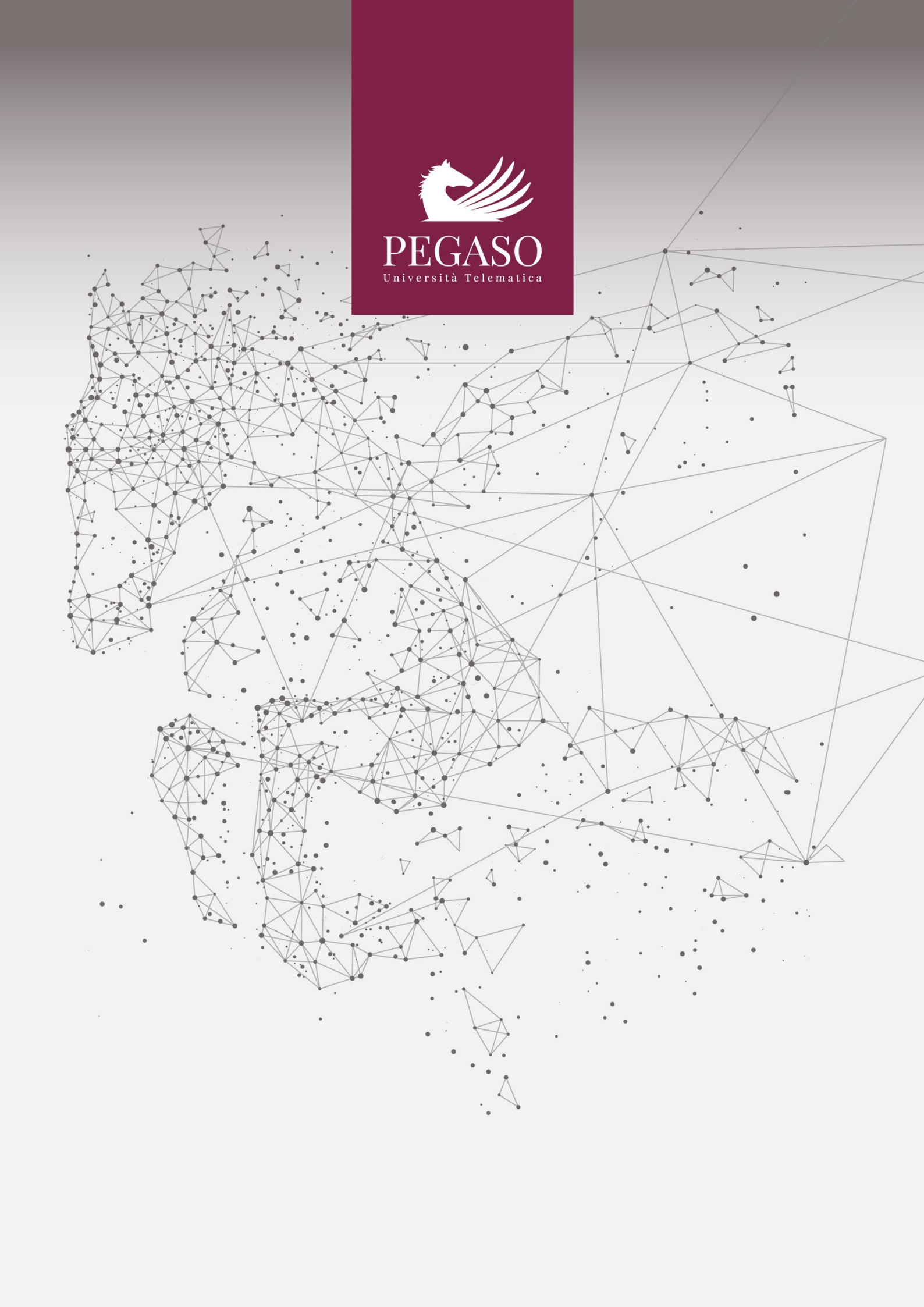

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE 3

2. STRUTTURE DECISIONALI 4

2.1 ESPRESSIONI BOOLEANE E OPERATORI RELAZIONALI 5
2.2 STRUTTURE DECISIONALI NIDIFICATE 8

3. LOGICA BOLEANA 11

4. STRUTTURE ITERATIVE 14

4.1 CICLO WHITE 15
4.2 CICLO FOR 17

BIBLIOGRAFIA 21

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1.Introduzione

In questa dispensa di accompagnamento alla video-lezione, si introducono le strutture decisionali e le strutture iterative caratteristiche del linguaggio di programmazione Python.

Verranno fornite anche informazioni in merito alle espressioni e agli operatori logici della logica booleana, oltre che diversi esempi relativi agli argomenti trattati.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2.Strutture decisionali

Una **struttura di controllo** (o **struttura decisionale**) è uno schema logico che controlla l'ordine in cui vengono eseguite delle istruzioni. Tale struttura differisce dalla **struttura sequenziale** che rappresenta un insieme di istruzioni che vengono eseguite nell'ordine in cui compaiono.

Anche se molto utilizzata, la struttura sequenziale non permette di risolvere alcuni tipi di problemi. A titolo di esempio, si pensi ad un software che deve applicare l'aliquota IRPEF in base allo scaglione del reddito. L'applicazione dell'aliquota corretta dipenderà proprio dall'ammontare del reddito

In Figura 1, si riporta il simbolo utilizzato per le strutture decisionali all'interno dei **diagrammi di flusso**. Il **blocco romboidale** indica il test (o verifica) di una condizione. Tale test potrà avere solo due risultati possibili: vero o falso. Il programma, quindi, si comporterà diversamente (eseguirà istruzioni diverse) in base alla condizione.

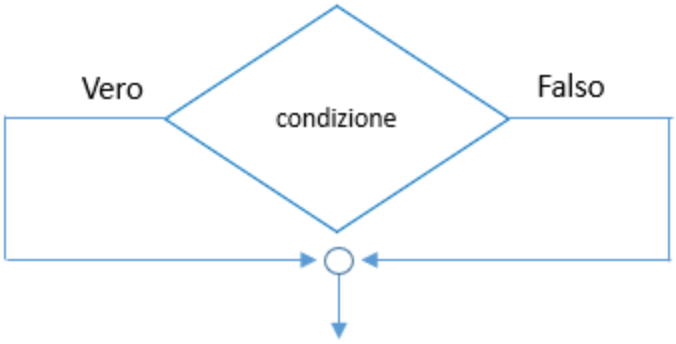


Figura 1. Simbolo delle strutture decisionali utilizzato nei diagrammi di flusso

Nell'esempio che segue, alla variabile *a* viene assegnato il valore 9 e come output del programma si vuole sapere se la variabile è positiva o negativa. La condizione della struttura di controllo sarà, dunque, a > 0. Se la condizione sarà vera la variabile *a* sarà positiva, altrimenti sarà negativa.

In Figura 2, si riporta il diagramma di flusso corrispondente al semplice programma appena descritto (il diagramma di flusso è stato realizzato con il software Raptor).

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

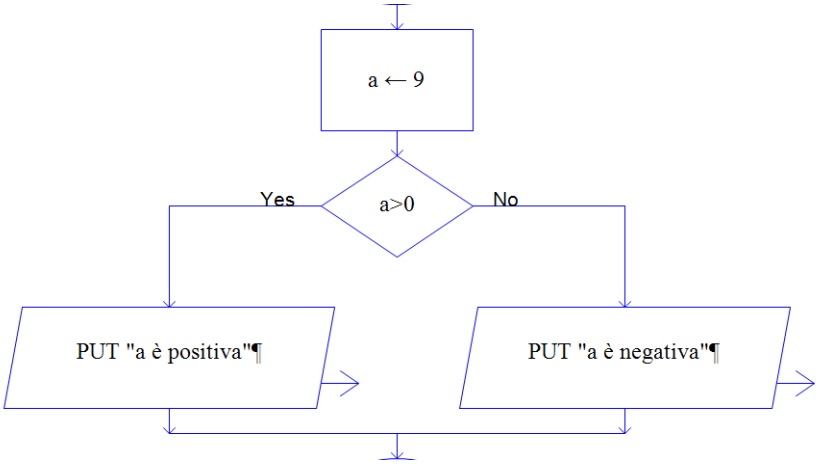


Figura 2. Diagramma di flusso del programma per determinare se una variabile è positiva o negativa

La struttura decisionale in Python si può tradurre con diverse istruzioni. La più semplice è l'**istruzione if** che permette di testare una condizione ed eseguire delle istruzioni solo se la condizione è vera. La sintassi dell'istruzione if in Python è la seguente:

```
if condizione:
    istruzione1
    istruzione2
```

La condizione può essere vera o falsa, ma le istruzioni istruzione1 e *istruzione2* verranno eseguite solo nel caso in cui la condizione fosse vera, altrimenti verranno saltate.

La sintassi di Python prevede che dopo la **keyword if** e la condizione vengano inseriti i **due punti :** e che dalla riga successiva inizi quello che viene chiamato **blocco di istruzioni**. Fondamentale ed indispensabile per il corretto funzionamento del programma è il rientro (spazio corrispondente ad una tabulazione orizzontale) utilizzato per le istruzioni del blocco. Il rientro serve, infatti, all'interprete Python per capire dove inizia e finisce il blocco di istruzioni.

2.1 Espressioni booleane e operatori relazionali

La condizione verificata dall'istruzione if rappresenta una **espressione booleana**, cioè un'espressione in cui è possibile utilizzare nei calcoli i concetti astratti di **vero** e **falso**.

L'espressione booleana verificata da un'istruzione if è formata solitamente da un **operatore relazionale**. L'operatore relazionale determina se esiste una determinata relazione tra due valori.

Gli operatori relazionali sono tipicamente **binari** (operano su due operandi) e sono compresi tra gli operandi (es. a > b).

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Nella Tabella 1 si riportano gli operatori relazionali previsti in Python.

Operatore	Significato	Esempio di uso
>	Maggiore di	a>b
<	Minore di	a<b
>=	Maggiore o uguale a	a>=b
<=	Minore o uguale a	a<=b
==	Uguale a	a==b
!=	Diverso da	a!=b

Tabella 1. Operatori relazionali previsti in Python

Nella condizione dell'istruzione if è importante l'utilizzo del **doppio uguale ==** per verificare l'uguaglianza tra due operandi. Non bisogna confondere il doppio uguale == con l'**operatore = di assegnazione.**

Un esempio di utilizzo corretto sia dell'operatore assegnazione sia dell'operatore di uguaglianza è riportato di seguito:

```
a=3
b=3
if a==b:
    print("a e b sono uguali")
```

Il risultato dell'elaborazione è:

```
a e b sono uguali
```

Si riporta ora un altro esempio di utilizzo dell'istruzione if con input e output. In particolare, si chiede in input la votazione conseguita e se tale votazione è superiore o uguale a 18, viene visualizzata a schermo la stringa "Complimenti! Esame superato!".

Il codice corrispondente all'esempio appena illustrato è il seguente:

```
a = int(input("Inserire la votazione conseguita: "))
if a>=18:
    print("Complimenti! Esame superato!")
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
if "claudio"=="Claudio":
    print("Le due stringhe sono uguali")
else:
    print("Le due stringhe sono diverse")
```

Il risultato dell'elaborazione per una votazione di esempio inserita in input pari a 28 è:

```
Inserire la votazione conseguita: 28
Complimenti! Esame superato!
```

Oltre all'istruzione if, per poter eseguire un blocco di istruzioni anche nel caso in cui la condizione fosse falsa, Python mette a disposizione l'istruzione **if-else**. Il formato dell'istruzione if-else è:

```
if condizione:
    blocco_vero
else:
    blocco falso
```

Si noti che le keyword **if** ed **else** devono essere allineate, che i due punti sono necessari anche dopo la keyword else e che anche le istruzioni nei blocchi devono essere allineate (in gergo tecnico si parla di **indentazione del codice**).

Grazie all'istruzione if-else è possibile tradurre il diagramma di flusso che permette di stabilire se un numero è positivo o negativo (Figura 2) in Python come segue:

```
if a>0:
    print("a è positivo")
else:
    print("a è negativo")
```

Particolare attenzione deve, poi, essere posta nel confronto tra stringhe. Ad esempio, le stringhe «claudio» e «Claudio» confrontate in un'istruzione if-else saranno diverse perché bisogna tener conto del minuscolo/maiuscolo. Il codice sorgente che dimostra tale confronto è:

Il risultato dell'elaborazione del codice sorgente risulta essere:

```
Le due stringhe sono diverse
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2.2 Strutture decisionali nidificate

Le istruzioni if-else possono essere **nidificate**, cioè il blocco istruzioni dei rami vero e falso di una istruzione if-else possono contenere a loro volta istruzioni if-else.

Un esempio molto semplice quanto istruttivo è il problema del trovare il maggiore tra 3 numeri memorizzati nelle variabili *a,b,c*. Il diagramma di flusso che risolve questo problema e riportato in Figura 3, prevede confronti consecutivi tra coppie di variabili.

Il ramo vero, così come il ramo falso del primo blocco if-else (con condizione a>b) contiene, a sua volta, un altro blocco if-else.

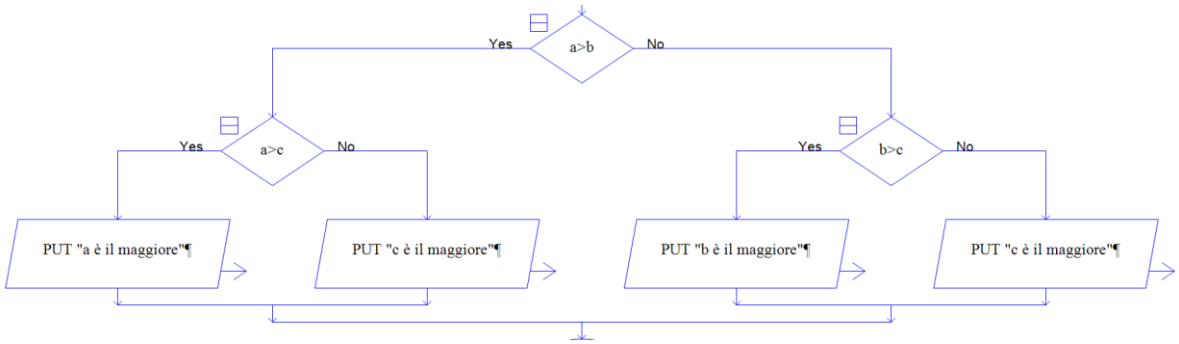


Figura 3. Diagramma di flusso per la risoluzione del problema del maggiore tra tre numeri contenuti in altrettante variabili (a,b,c)

Il codice Python corrispondente sarà:

```
if a>b:
    if a>c:
        print("a è il maggiore")
    else:
        print("c è il maggiore")
else:
    if b>c:
        print("b è il maggiore")
    else:
        print("c è il maggiore")
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Sempre in merito alle strutture decisionali nidificate, Python mette a disposizione una terza istruzione per le strutture decisionali, l'istruzione **if-elif-else**. Questa istruzione semplifica la scrittura di alcune strutture annidate e può includere più **elif**.

La sintassi dell'istruzione **if-elif-else** in Python è la seguente:

```
if condizione1:
    blocco
elif condizione2:
    blocco
elif condizione3:
    blocco
else:
    blocco
```

L'istruzione **if-elif-else** è molto utile in tuti quei problemi per cui ci sono da individuare delle fasce. In Figura 4 **Errore. L'origine riferimento non è stata trovata.** si riporta il diagramma di flusso che permette di conoscere il giudizio sulla base di una valutazione numerica. Supponendo che l'intervallo dei voti sia compreso tra 0 e 10 estremi inclusi, si verificano le seguenti condizioni successive:

- se il voto è maggiore o uguale a 9 (e implicitamente inferiore a 10), il giudizio corrispondente sarà **Ottimo**;
- altrimenti, se il voto non rispetta la condizione precedente, vorrà dire che il voto sarà compreso tra 0 e 9 (escluso). Si verificherà, quindi, se il numero sia maggiore o uguale a 8 (e implicitamente inferiore a 9) e in caso positivo il giudizio corrispondente sarà **Buono**;
- altrimenti, se il voto non rispetta la condizione precedente, vorrà dire che il voto sarà compreso tra 0 e 8 (escluso). Si verificherà, quindi, se il numero sia maggiore o uguale a 7 (e implicitamente inferiore a 8) e in caso positivo il giudizio corrispondente sarà **Discreto**;
- altrimenti, si verificherà se voto>=6: in caso positivo il giudizio sarà **Sufficiente** e in caso negativo (voto compreso tra 0 e 6 escluso) sarà **Insufficiente**.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

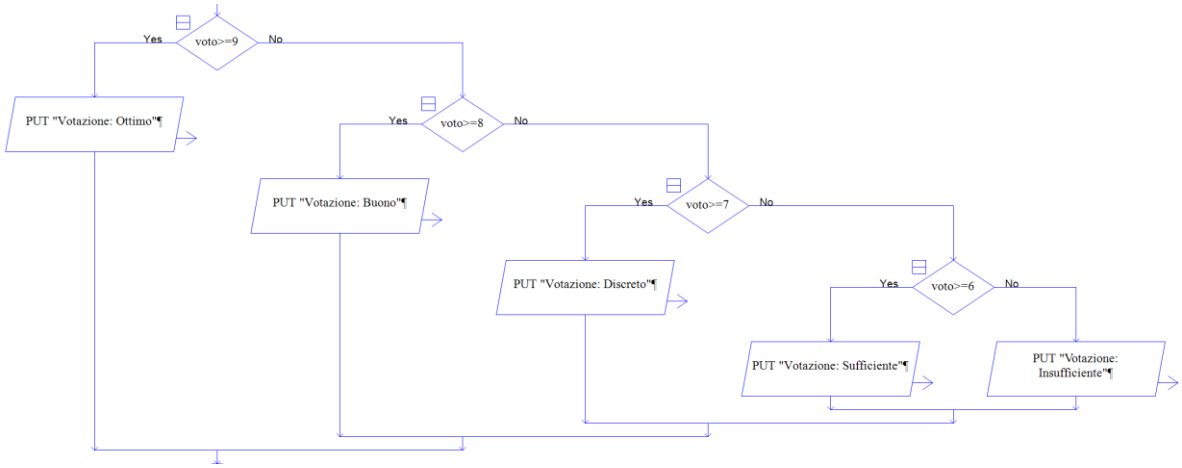


Figura 4. Struttura corrispondente all'istruzione if-elif-else per il problema di assegnazione del giudizio nota la votazione numerica

La traduzione del diagramma di flusso precedente, in Python diventa:

```
if voto>=9:
    print("Votazione: Ottimo")
elif voto>=8:
    print("Votazione: Buono")
elif voto>=7:
    print("Votazione: Discreto")
elif voto>=6:
    print("Votazione: Sufficiente")
else:
    print("Votazione: Insufficiente")
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3.Logica boleana

Spesso c'è la necessità di utilizzare espressioni booleane più complesse nelle condizioni delle strutture decisionali. A tale proposito, si utilizzano gli **operatori logici**:

- gli operatori **and** e **or** sono operatori binari che agiscono su due operandi booleani (quindi operandi che possono assumere valore vero o falso);
- l'operatore **not** è, invece, un operatore unario, cioè un operatore che agisce su un solo operando booleano.

Gli operatori logici sono completamente definiti dalle **tavole della verità** ad essi associati. Le tavole della verità permettono di conoscere il risultato dell'operatore logico applicato agli operandi per tutte le possibili combinazioni di valore logico di questi ultimi.

Segue la tavola della verità dell'operatore logico binario **and**. Ai fini di una maggiore comprensione, si presenta anche un esempio in cui si verifica la condizione di appartenenza di un punto di coordinate (x,y) al I quadrante dello spazio cartesiano.

Cond. A	Cond. B	A and B	Es. cond. A x>0	Es. cond. B y>0	Es. A and B x>0 e y>0
Falso	Falso	Falso	Falso (x=-1)	Falso (y=-1)	Falso
Falso	Vero	Falso	Falso(x=-1)	Vero (y=1)	Falso
Vero	Falso	Falso	Vero (x=1)	Falso (x=-1)	Falso
Vero	Vero	Vero	Vero (x=1)	Vero (y=1)	Vero

Segue, ora, la tavola della verità dell'operatore logico binario **or**. Ai fini di una maggiore comprensione, si presenta un esempio in cui si verifica la condizione di appartenenza di un punto di coordinate (x,y) al I o al II o al IV quadrante dello spazio cartesiano.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Cond. A	Cond. B	A and B	Es. cond. A x>0	Es. cond. B y>0	Es. A and B x>0 e y>0
Falso	Falso	Falso	Falso (x=-1)	Falso (y=-1)	Falso
Falso	Vero	Vero	Falso(x=-1)	Vero (y=1)	Vero
Vero	Falso	Vero	Vero (x=1)	Falso (x=-1)	Vero
Vero	Vero	Vero	Vero (x=1)	Vero (y=1)	Vero

Infine, si riporta la tavola della verità dell'operatore logico unario **not** che inverte il valore logico della condizione. Ai fini di una maggiore comprensione, si presenta un esempio in cui si verifica se un numero è negativo.

Cond. a	not a	Es. cond. a x>0	Es. not a X<0
Falso	Vero	Falso (x=-1)	Vero
Vero	Falso	Vero (x=1)	Falso

Gli operatori logici sono molto utili anche per verificare l'appartenenza di un numero ad un determinato intervallo numerico (esempio: *x >= 10 and x <= 20*) o per determinare se lo stesso numero è al di fuori di un intervallo (*esempio: x < 10 or x > 20*).

Inoltre, l'utilizzo degli operatori logici, spesso, semplifica anche il codice. A titolo di esempio, i due codici seguenti sono equivalenti per funzionalità, ma utilizzano un diverso numero di strutture decisionali.

Codice 1:

```
if x>=10:
    if x<=20:
        print("il valore è nell'intervallo")
```

Codice 2:

```
if x>=10 and x<=20:
    print("il valore è nell'intervallo")
```

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

In Python, oltre ai tipi di variabili int, float e str, esiste un tipo di variabile booleana, il tipo **bool** che può assumere due valori logici: *True* o *False*. In base all'utilizzo che se ne fa nel programma, una variabile booleana può essere chiamata in gergo tecnico **flag** (bandiera). Il flag è una variabile che segnala quando viene verificata una condizione in un programma:

- Flag a False → condizione non verificata;
- Flag a True → condizione verificata.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Strutture iterative

I simboli utilizzati nei diagrammi di flusso per le strutture iterative sono due e differiscono per la posizione del test condizionale di continuazione/fine ciclo. In particolare, se la condizione viene verificata successivamente al blocco di istruzioni (in questo caso il blocco di istruzioni verrà eseguito almeno una volta) si parla di **struttura post-condizionale** (Figura 5), altrimenti, se la condizione viene verificata prima del blocco di istruzioni (in questo caso il blocco di istruzioni potrebbe anche non essere mai eseguito) si parla di **struttura iterativa pre-condizionale** (Figura 6).

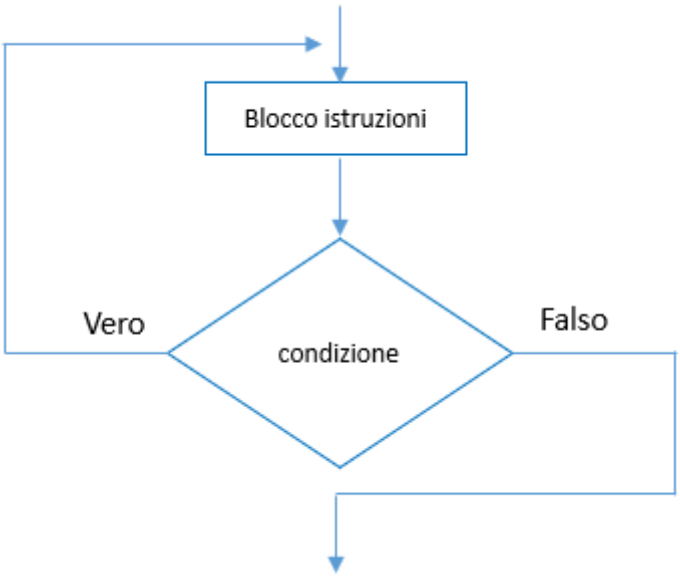


Figura 5. Simbolo della struttura iterativa pre-condizionale

Ad ogni modo, convenzionalmente, entrambe le strutture iterative "ciclano per vero", cioè eseguono/rieseguono il blocco di istruzioni nel caso in cui la condizione sia vera.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

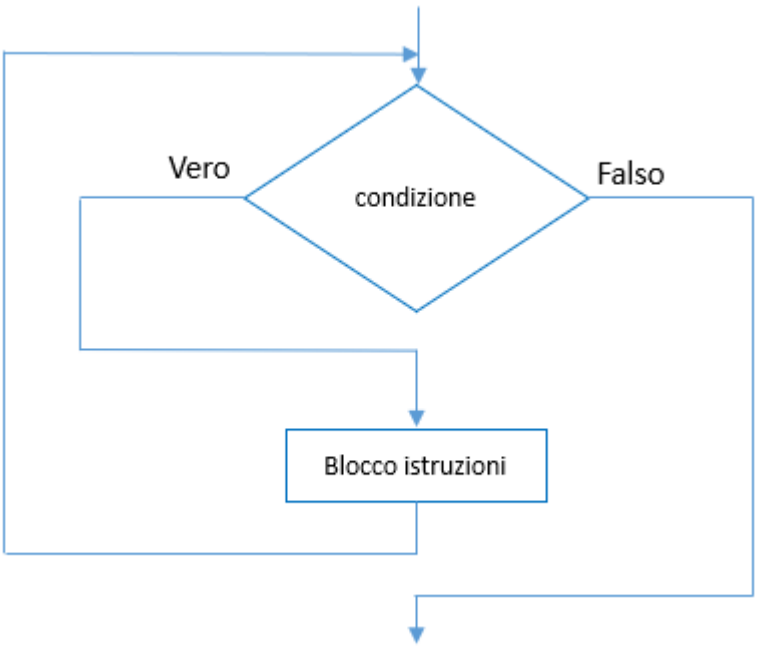


Figura 6. Simbolo della struttura iterativa pre-condizionale

4.1 Ciclo white

In Python, il **ciclo while** è conosciuto anche come **pre-test loop** (o **ciclo pre-condizionale**). Prima di eseguire un'iterazione si verifica la condizione. Come anticipato in precedenza, il blocco di istruzioni non verrà mai eseguito se la condizione è falsa in partenza.

Se all'interno del blocco di istruzioni del ciclo non ci sono istruzioni che direttamente o indirettamente modificano il valore logico della condizione di ciclo, c'è il rischio di ciclo infinito. Il formato generale del ciclo while in Python è:

```
while condizione:
    blocco_istruzioni
```

Il ciclo while continua ad eseguire il blocco di istruzioni finché la condizione è vera. Un semplice esercizio con il ciclo while potrebbe essere quello di visualizzare a schermo i numeri da 1 a 10. Si inizializza la variabile i=1 e si imposta un ciclo while in cui si itera finché il valore della variabile i è minore di 10. Nel blocco di istruzioni, si andrà a stampare il valore della variabile i e, successivamente, ad incrementarne il valore di 1. Il codice che esegue tale conteggio è il seguente:

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
i=1
while i<=10:
    print(i)
    i=i+1
```

Il risultato dell'elaborazione risulterà essere:

```
1
2
3
4
5
6
7
8
9
10
```

Si presenta ora un esercizio con l'utilizzo di un flag per il controllo di un ciclo while. Il flag *termina* indicherà quando l'inserimento di un insieme di nomi dovrà terminare:

- se *termina = False* il ciclo deve continuare (inserimento di nomi deve continuare);
- se *termina = True* il ciclo si interrompe.

Il valore iniziale del flag in questo caso deve essere impostato a False. Il codice seguente continua ad iterare finché non viene inserita in input la stringa "fine".

```
termina = False
while not termina:
    nome = input("Inserire nominativo: ")
    if(nome == "fine"):
        termina=True
    else:
        print("Benvenuto/a", nome)
```

Il risultato dell'elaborazione a seguito dell'inserimento di alcuni nomi di fantasia è:

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
Inserire nominativo: Anna
Benvenuto/a Anna
Inserire nominativo: Pasquale
Benvenuto/a Pasquale
Inserire nominativo: Lorenzo
Benvenuto/a Lorenzo
Inserire nominativo: Rosa
Benvenuto/a Rosa
Inserire nominativo: fine
```

4.2 Ciclo for

In Python, il **ciclo for** è conosciuto anche come **count-controlled loop** (o ciclo controllato numericamente). Il ciclo for itera per un numero definito di volte. Il formato generale dell'istruzione for è:

```
for variabile in [val1, val2, etc]:
    statements
```

Ad ogni iterazione del ciclo, la variabile del for assumerà uno dei valori dell'insieme ordinato riportato tra parentesi quadre. Il numero di iterazioni sarà dunque pari al numero di elementi nell'insieme di valori.

Ad ogni iterazione verranno eseguite le istruzioni comprese nel blocco. Si riporta il codice per un esempio di conteggio dei numeri da 1 a 5 con l'istruzione for:

```
for i in [1,2,3,4,5]:
    print(i)
```

che produce come risultato:

```
1
2
3
4
5
```

Per semplificare la scrittura dell'istruzione *for* soprattutto per l'insieme dei valori da inserire dopo la **keyword in**, è possibile utilizzare la funzione **range**() che restituisce un **oggetto iterabile.** Un oggetto iterabile contiene una sequenza di valori che possono essere utilizzati per iterare. Le caratteristiche della funzione range in base al numero degli argomenti passati alla funzione stessa sono:

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- se viene indicato **un solo argomento** questo viene usato come limite superiore escluso (di default limite inferiore è uguale a zero). Esempio di codice:

```
for i in range(10):
    print(i)
```

che permette di visualizzare a schermo il seguente risultato:

```
0
1
2
3
4
5
6
7
8
9
```

- se vengono indicati **due argomenti** questi vengono usati rispettivamente come limite inferiore (incluso) e limite superiore (escluso). Esempio di codice:

```
for i in range(1,10):
    print(i)
```

che permette di visualizzare a schermo il seguente risultato:

```
1
2
3
4
5
6
7
8
9
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- se vengono indicati **tre argomenti**, essi corrisponderanno rispettivamente al limite inferiore (incluso), limite superiore (escluso) e **step value** (incremento). Esempio di codice:

```
for i in range(1,10,2):
    print(i)
```

che permette di visualizzare a schermo il seguente risultato:

```
1
3
5
7
9
```

La funzione range con tre argomenti può prevedere anche limite inferiore, limite superiore e incremento negativo così come riportato nel seguente esempio di codice sorgente:

```
for i in range(1,-10,-1):
    print(i)
```

che permette di visualizzare a schermo il seguente risultato:

```
1
0
-1
-2
-3
-4
-5
-6
-7
-8
-9
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

A volte il programmatore non conosce esattamente il numero di iterazioni che dovranno essere eseguite.

È possibile che l'utente fornisca direttamente a run-time il numero di iterazioni. In questi casi, la funzione range viene chiamata all'interno dell'istruzione for utilizzando come limite/i una o più variabili come riportato nel codice sorgente di esempio:

```
end = int(input("Fino a che numero vuoi contare? "))
for i in range(1,end+1):
    print(i)
```

che visualizza a schermo:

```
1
2
3
4
5
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Gaddis, T. (2016). Introduzione a Python. Ediz. mylab. Con espansione online. Pearson.
- Python (2021). https://www.python.org/

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).